



C프로그래밍 (CSE2003)

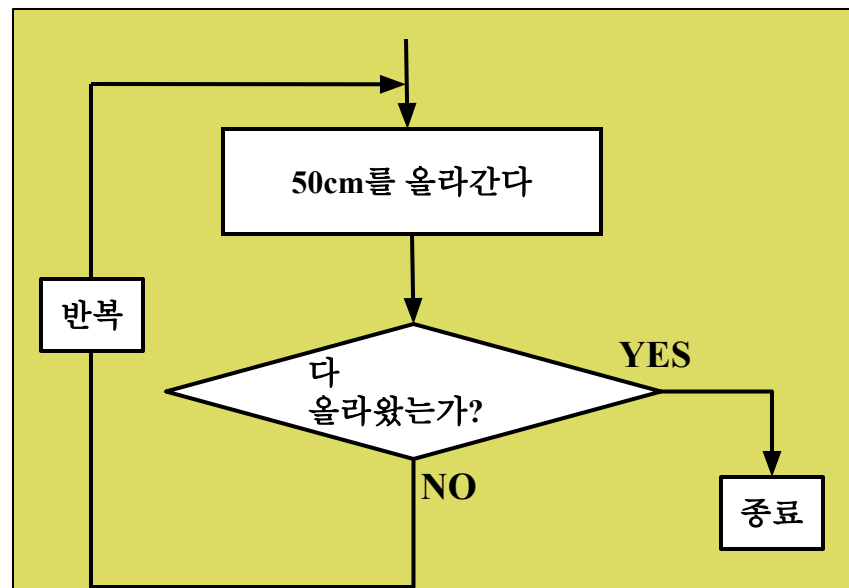
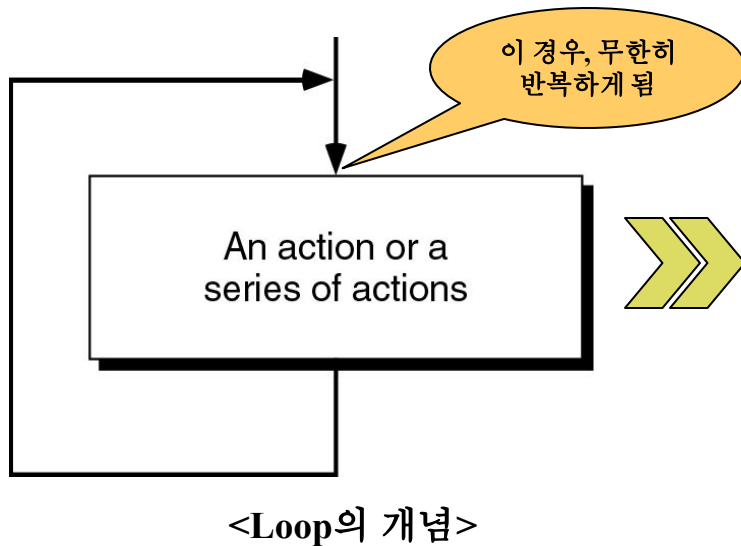
6. Repetition

반복문(loop)이란?

- 반복문(loop)은 어떤 조건에 도달할 때까지 특정 상태(condition)를 만족하는 동안 동일한 작업을 반복하여 수행할 수 있도록 해준다.

우물의 깊이는 3m이고 달팽이는 하루에 50cm를 올라갑니다.

만약 달팽이가 미끄러지지 않는다면 달팽이는 몇 일만에 우물을 벗어날 수 있을까요?

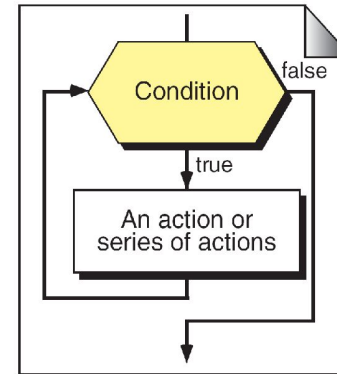


Pretest and Post-test Loops

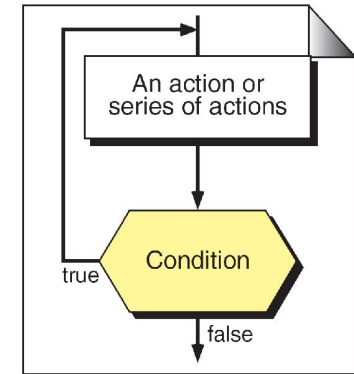
- Pretest loop

- loop control expression 체크
- **true** □ loop안의 statements를 실행 & loop의 첫 단계로 돌아감
- **false** □ loop를 중단

Condition에 따라서 statement가 실행되지 않을 가능성이 있다.



(a) Pretest Loop

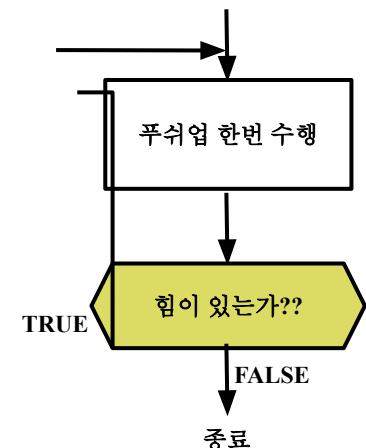
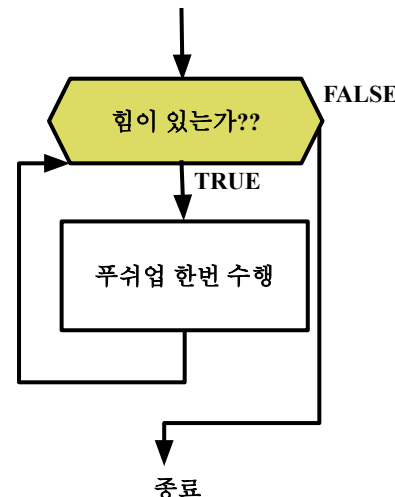


(b) Post-test Loop

- Post-test loop

- loop 안의 statements를 먼저 실행
- loop control expression 체크
- **true** □ loop의 첫 단계로 돌아감
- **false** □ loop를 중단

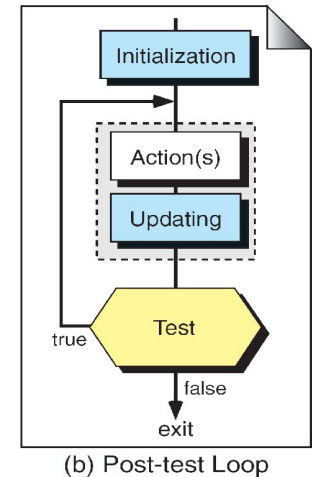
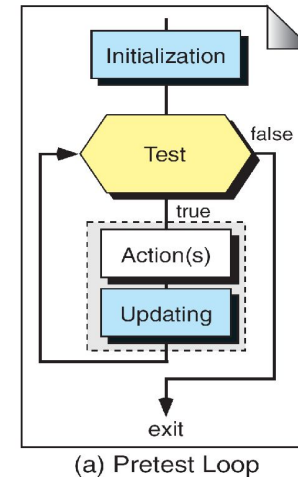
Condition에 상관없이 statement는 최소한 1회 이상 실행된다.



Initialization & Updating

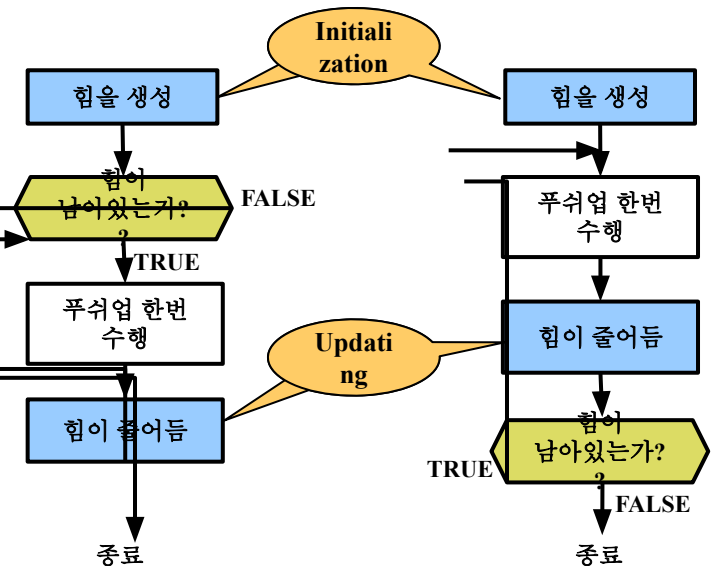
• Initialization

- Loop가 시작되기 전에,
loop initialization(초기화)가 필요
- 일반적으로 제어변수(control variable)값을 할당



• Updating

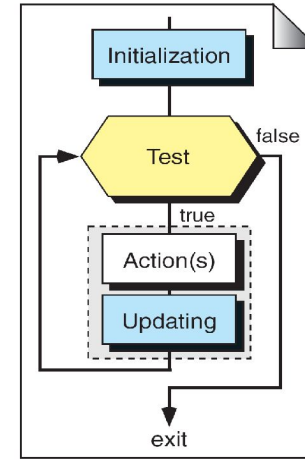
- loop body에서 condition의 변화가 필요
 - 무한 루프를 피하기 위해서
- Loop Condition에 변화를 줌으로서, 제어조건을 FALSE로 바꿈
 - 루프 종료



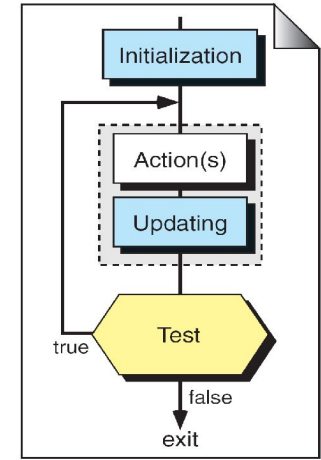
Event & Counter - controlled Loops

■ Event-Controlled Loops

- loop control expression 의 값을 true 에서 false로 바꾸는 어떤 **event**가 발생
- 반복횟수를 알 수 없음.
 - 별도의 count변수 필요



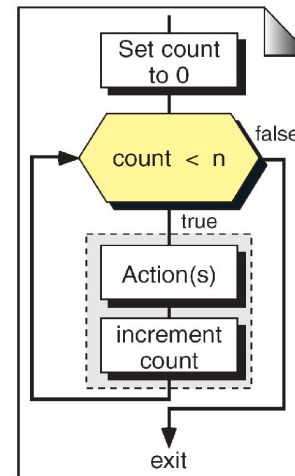
(a) Pretest Loop



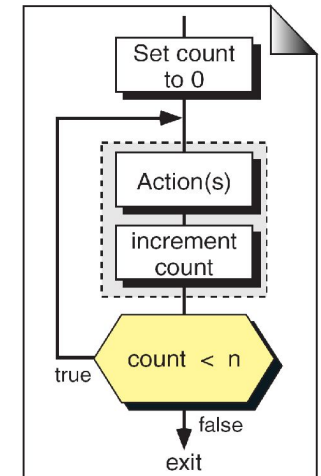
(b) Post-test Loop

■ Count-Controlled Loops

- 반복횟수를 condition으로 사용
- count 변수의 값을 증가 또는 감소시키면서 원하는 횟수만큼 반복했을 때 중지



(a) Pretest Loop



(b) Post-test Loop

Event & Counter - controlled Loops

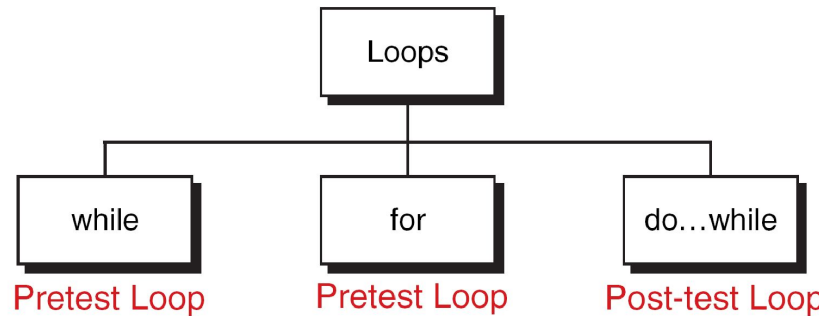
■ Loop Comparison

- 반복을 n 번 수행한다고 할 때, Pretest Loop과 Post-test 반복횟수 비교

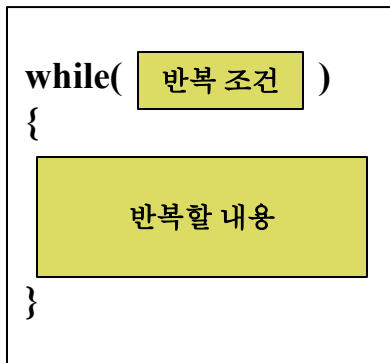
	Pretest Loop	Post-test Loop
Initialization	1	1
Number of tests	$n+1$	n
Action execute	n	n
Updating executed	n	n
Minimum iteration	0	1

C에서의 반복문들

<C loop construct>

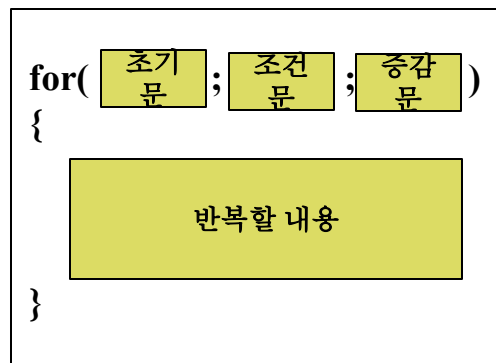


<while문>



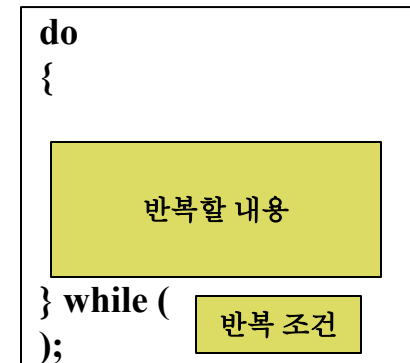
“반복 조건”이 만족되는 동안
“반복할 내용”을 반복
실행한다.

<for문>



최초 “초기문”을 실행하고, “조건문”을 만족하면
“반복할 내용”과 “증감문”을 차례로 실행한다.

<do while문>



일단 루프를 한번 실행 후,
“반복조건”이 만족되는 동안
“반복할 내용”을 반복실행한다.

The *while* Loop

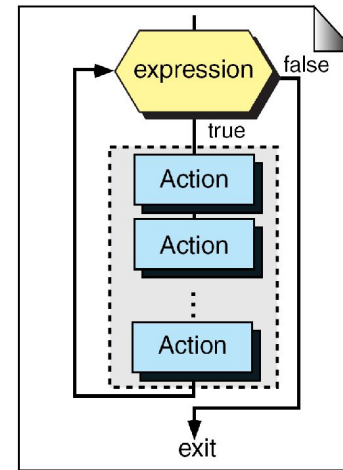
■ while 문 구조

기본원리

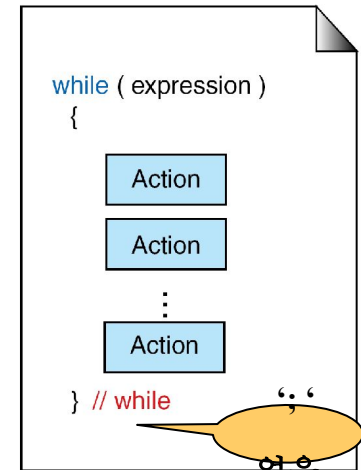
특정 조건을 주고 그 조건이 만족될 때까지
계속해서 반복을 시킨다

```
while( i < 10 )  
{  
    printf("Hello World!\n");  
    i++;  
}
```

[while문의 몸체(body)]
한 문장으로 구성되는 경우,
블록형태를 취하지 않아도 됨



(a) Flowchart



(b) C Language

■ while 문 실행순서

- 조건문인 $i < 10$ 을 검사하여 i 가 조건을 만족하면(true) 반복문의 몸체(body)에 당하는 문장이나 블록을 실행한다.
- 모든 몸체를 실행하면 다시 조건문을 검사하는 첫 번째 과정을 반복한다. 첫 번째 과정에서 조건을 만족하지 않으면(false) while문을 종료한다.

The *while* Loop

예제 프로그램 – while loop

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int height = 0;
6     int days = 0;
7     int depth;
8
9     printf("Input the depth of well(cm) : ");
10    scanf("%d", &depth);
11
12    while(height < depth)
13    {
14        height = height + 50;
15        days = days + 1;
16    }
17
18    printf("Total days: %d\n", days);
19 }
```

```
[root@mclab c-lang]# vi chap6-1.c
[root@mclab c-lang]# gcc -o chap6-1 chap6-1.c
[root@mclab c-lang]# ./chap6-1
Input the depth of well(cm) : 300
Total days: 6
[root@mclab c-lang]#
```

달팽이 우물 탈출 일수 구하는 프로그램

1) 우물의 높이에 미치지 못하면

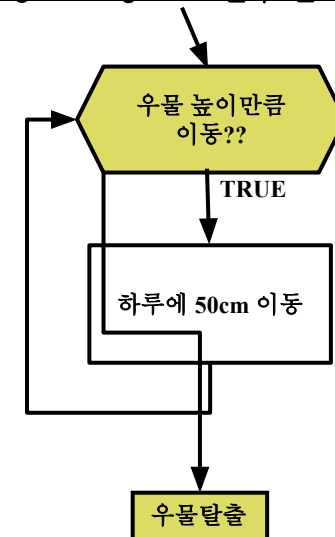
(height < depth)

□ while 문의 내용을 반복 실행

2) 우물의 높이에 도달하면

(height >= depth)

□ while문 종료 & 총 소요일수 출력



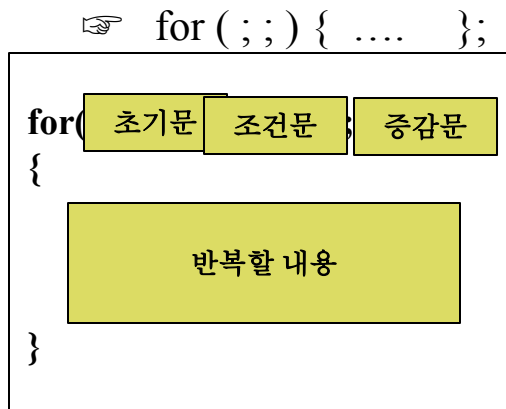
실습 20

실습 21

The *for* Loop

- for문 구조

- 초기문 (Initialization), 조건문 (Limit-test expression), 증감문 (Updating expression)
- 각 구문별로 세미콜론(;) 으로 구분
 - 괄호() 안에 세미콜론은 반드시 필요하다.
 - 괄호() 안에 각 세 부분은 생략되어도 문법 오류는 발생하지 않음



```
for ( i=0 ; i<10 ; i++ )  
{  
  
    printf("Hello World!\n");  
  
}
```

[for문의 몸체(body)]

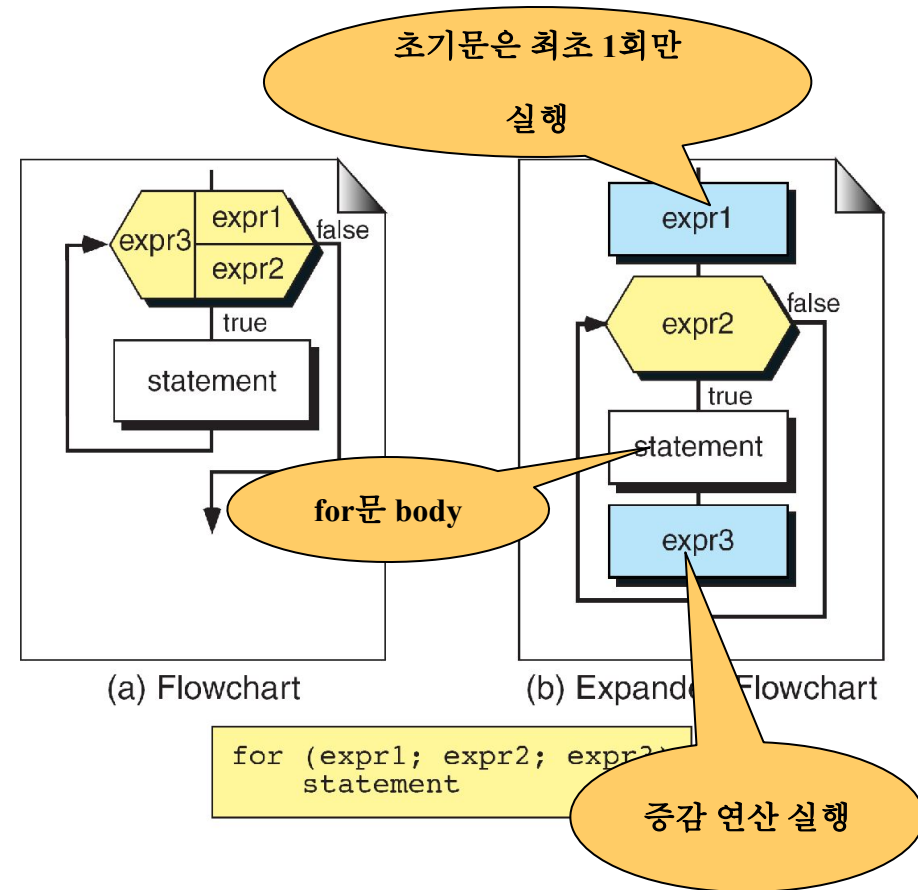
한 문장으로 구성되는 경우,
블록형태를 취하지 않아도 됨

The *for* Loop

• for 문의 실행순서

- ① 초기화(expr1)를 실행한다.
- ② 조건검사(expr2)를 실행한다.
- ③ 조건이 참이면
loop body (statement)를 수행한다.
expr3을 수행하고 ②로 돌아간다.
- ④ 조건이 거짓이면 for문을 종료한다.

```
for ( expr1 ; expr2 ; expr3 )  
{  
    statement  
}
```



The *for* Loop

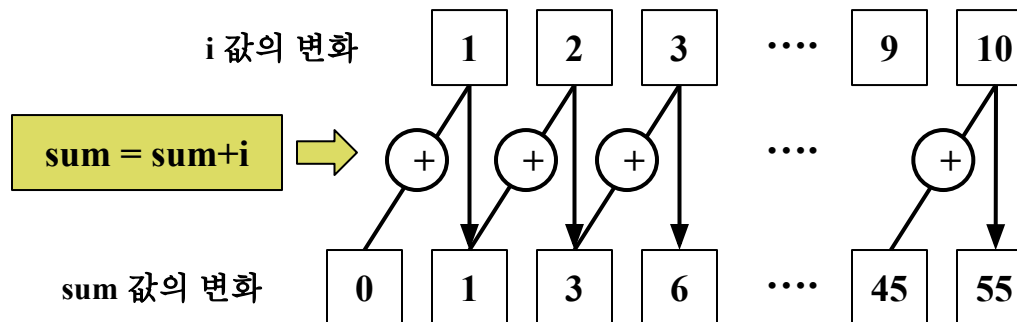
예제 프로그램 - for loop

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int i, sum=0;
6
7     for(i=0; i<=10; i++)
8         sum+=i;
9
10    printf("sum=%d\n", sum);
11    return 0;
12 }
```

```
[root@mclab c-lang]# vi chap6-2.c
[root@mclab c-lang]# gcc -o chap6-2 chap6-2.c
[root@mclab c-lang]# ./chap6-2
sum=55
[root@mclab c-lang]#
```

1부터 10까지 더하는 프로그램

□ i 를 1부터 10까지 증가시키면서 sum에 더해준다.



The *for* Loop

- 예제 프로그램 - nested for loop

```
#include <stdio.h>

int main (void)
{
    int i, j;

    for(i=2; i<4; i++)
    {
        for(j=1; j<10; j++)
            printf("%d * %d = %d\n", i, j, i*j);
    }

    return 0;
}
```

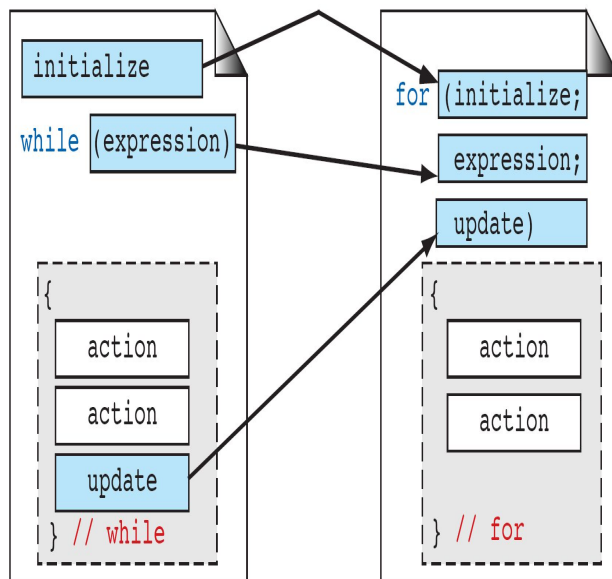
구구단 2 단 출력하는 프로그램

□ for 문이 중첩되어 사용되고 있다.

```
@cspro:~/lec6$ gcc -o chap6-3 chap6-3.c
@cspro:~/lec6$ ./chap6-3
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
@cspro:~/lec6$
```

The *for* Loop

- **for**와 **while** loop의 비교
 - for loop은 while loop과 동일한 역할을 수행하지만 readability 가 좋으며 counting loop에 자연스럽다.



<while loop>

```
i=1;
sum=0;
while(i<=20)
{
    scanf("%d",&a);
    sum += a;
    i++;
} //while
```

<for loop>

```
sum = 0;
for (i = 1; i <=20 ; i++)
{
    scanf("%d",&a);
    sum += a;
} // for
```

거듭제곱 구하기

양의 정수 밑(base)과 지수(exponent)를

```
#include <stdio.h>

int powers(int base, int exp);

int main (void)
{
    int base, exp;

    printf("base : ");
    scanf("%d", &base);

    printf("exponent : ");
    scanf("%d", &exp);

    printf("%d^%d=%d\n", base, exp, powers(base, exp));

    return 0;
}

int powers(int base, int exp)
{
    int result = 1;
    int i;

    if(base < 1 || exp < 0)
        result = 0;
    else
        for(i=1; i<=exp; i++)
            result *= base;

    return result;
}
```

- 입력 받아 거듭제곱을 구하는 프로그램
- 1) 사용자로부터 밑과 지수를 입력 받는다.
 - 2) 만약 밑이 1보다 작거나 지수가 음수이면 결과값은 0으로 한다.
 - 3) powers() 함수의 for 문에서 입력 받은 지수만큼 루프를 돌게 되어 있다.
 - 4) 계산한 결과를 반환하여 main 함수에서 출력한다.

```
spro:~$ gcc -o powers powers.c
spro:~$ ./powers
base : 4
exponent : 2
4^2=16

spro:~$ ./powers
base : 5
exponent : 0
5^0=1
```

실습 22

실습 23